

Mini Debugger – Technical Report

Chirag Suthar
2025MCS2105

Gandem Nitin D.
2025MCS2152

1 Introduction

This technical report explains the internal design, algorithms, and system programming concepts used to build a **Mini Debugger** for UNIX-like operating systems.

The Mini Debugger provides a simplified but functionally accurate model of a real debugger such as `gdb`. Instead of focusing on source-level debugging, the project emphasizes **low-level process control and debugging mechanisms provided by the operating system**.

The debugger demonstrates the following core concepts:

- Process tracing using `ptrace()`
- Tracer–tracee process relationship
- Program execution control (`run`, `continue`, `step`, `break`, `delete`)
- Software breakpoint implementation using INT3 (0xCC)
- Breakpoint lifecycle handling
- CPU register inspection
- Signal handling using SIGTRAP
- Debugger state management
- Graceful error handling

The Mini Debugger is implemented in modular C source files:

- `main.c` – Interactive command-line interface (REPL)
- `include/debugger.c` – Process control and breakpoint handling
- `include/breakpoint.c` – Breakpoint table and memory patching
- `include/debug.h` – Debugger data structures
- `include/breakpoint.h` – Breakpoint APIs
- `tests/...` – Test programs

2 System Architecture Overview

The Mini Debugger follows a **control–observe–resume execution model**. The debugged program executes only when explicitly allowed by the debugger.

- User Command → Debugger Logic → `ptrace()` → Kernel → Tracee

Debugger State Machine

NOT_STARTED -> STOPPED -> RUNNING -> STOPPED -> EXITED
--

3 Tracer–Tracee Model

The debugger is built using the `ptrace` tracer–tracee relationship.

- Tracer: Debugger (parent)
- Tracee: Debugged program (child)

```
ptrace(PTRACE_TRACEME);
```

This enables execution control, register inspection, memory access, and signal handling.

4 Command-Line Interface (REPL)

Supported Commands

Command	Description
run ./program	Start program under debugger
continue	Resume execution
step	Execute one instruction
break hexaddr	Set breakpoint
delete hexaddr	Remove breakpoint
regs -s	Print stack registers
regs -g	Print general registers
regs	Print all registers
status	Show process state
help	Show command list
quit	Exit debugger

5 Program Execution Control

5.1 Program Launch (run)

1. `fork()`
2. Child calls `ptrace(PTRACE_TRACEME)`
3. Child executes program using `execvp()`
4. Parent waits using `waitpid()`

5.2 Continue Execution (continue)

```
ptrace(PTRACE_CONT);
```

Execution continues until a breakpoint or program exit.

5.3 Single-Step Execution (step)

```
ptrace(PTRACE_SINGLESTEP);
```

6 Software Breakpoint Implementation

6.1 Breakpoint Data Structure

Each breakpoint stores:

- Address
- Original instruction
- Active state

6.2 Setting a Breakpoint

1. Validate address
2. Read instruction using `PTRACE_PEEKDATA`
3. Replace LSB with `0xCC`
4. Write using `PTRACE_POKEDATA`

6.3 Breakpoint Hit Handling

- `SIGTRAP` is raised
- `RIP` points to `addr + 1`
- Restore original instruction
- Rewind `RIP`
- Single-step original instruction
- Reinsert breakpoint

6.4 Removing a Breakpoint

Original instruction is restored and breakpoint entry removed.

7 Logical Instruction Pointer Correction

`RIP` is adjusted for display purposes only to reflect the true breakpoint address.

8 Register Inspection

Registers are retrieved using:

```
ptrace(PTRACE_GETREGS);
```

Supported Registers

- `RIP`, `RSP`, `RBP`
- `RAX`, `RBX`, `RCX`, `RDX`
- `RSI`, `RDI`
- `R8–R15`

Registers are shown only when the program is STOPPED.

9 Signal Handling

9.1 SIGTRAP

Generated on breakpoint hits and single-step completion.

9.2 Program Exit

- Detected using `WIFEXITED`
- State set to `EXITED`

9.3 Other Signals

- WIFSIGNALED: EXITED
- WIFSTOPPED: STOPPED

10 Process State Reporting

The `status` command reports:

- NOT_STARTED
- RUNNING
- STOPPED
- EXITED

11 Error Handling and Robustness

All critical system calls are checked:

- `fork()`
- `ptrace()`
- `waitpid()`
- `execvp()`

Safety guarantees include instruction restoration and no memory modification while running.

12 Design Choices and Simplifications

Feature	Support
Instruction-level debugging	Supported
Software breakpoints	Supported
Multiple breakpoints	Supported
Register inspection	Supported
Source-level debugging	Not supported
Symbol resolution	Not supported
Multi-threaded debugging	Not supported

13 Testing and Verification

Basic Execution

```
run ./test
continue
```

Breakpoints

```
run ./test
break 0x40113e
continue
```

Single-Stepping

```
step
step
```

Breakpoint Removal

```
run ./test
break 0x40113e
continue
delete 0x40113e
continue
```

14 Conclusion

The Mini Debugger demonstrates how low-level UNIX debuggers work using:

- `ptrace()`
- `fork()`
- `execvp()`
- `waitpid()`
- CPU registers and signals

It provides a strong educational foundation in operating systems and debugging internals.

15 Git Commit History

```
* hobbb1t31@Smaug:~/Desktop/minimal-debugger/src$ for author in $(git log --format="%an" | sort -u);
  echo "==== Commits by $author ====="
  git log --author="$author" \
    --graph --all \
    --pretty=format:"%h %ad %s" \
    --date=local
  echo
done
==== Commits by chirag312001 =====
* 43b6d93 Tue Dec 16 15:56:18 2025 completed
* 98f7e94 Tue Dec 16 13:31:16 2025 process status, regs
* 23c22a5 Tue Dec 16 11:55:46 2025 asd
* df25dbb Mon Dec 15 23:21:29 2025 user manual added
* 7a7ccc2 Mon Dec 15 23:10:28 2025 technical report added
* 637e0fb Mon Dec 15 23:02:20 2025 my part done
* c4d3d8e Mon Dec 15 22:59:43 2025 adding breakpoints from CLI
* bb175d1 Sun Dec 14 22:37:17 2025 Breakpoint static
* 594769f Sat Dec 13 16:42:16 2025 solving
* be11404 Sat Dec 13 16:33:24 2025 bugs resolving
* 7861792 Sat Dec 13 03:38:19 2025 Dev A half work done breakpoints sets and inspection
* 9ee9f7a Fri Dec 12 16:59:58 2025 Done with 1st part moving to second
* 7b6d50b Thu Dec 11 14:14:09 2025 Strating
o hobbb1t31@Smaug:~/Desktop/minimal-debugger/src$
```

Figure 1: Git commits by Chirag

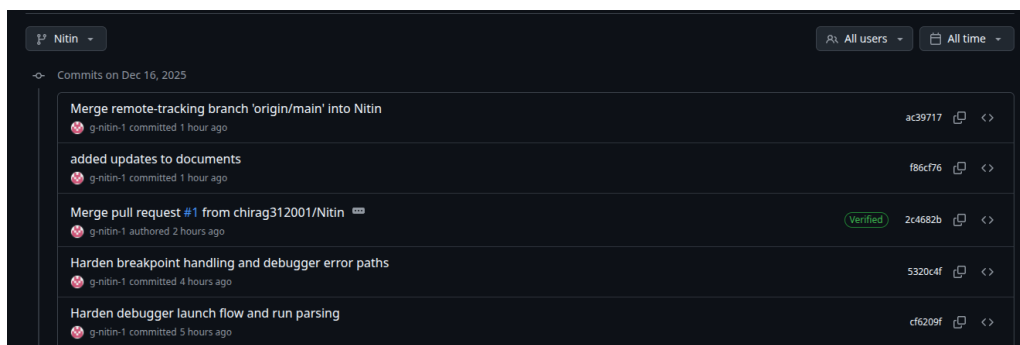


Figure 2: Git commits by Nitin